Progetto di High Performance Computing 2024/2025

Giovanni Tentelli, matr. 0000921144
20/09/2025

Introduzione

In questo progetto ho modificato il codice prodotto dal Prof. Marzolla per adattarlo al calcolo in parallelo con OpenMP e CUDA. Spiegherò in dettaglio le scelte utilizzate per ogni approccio compresi di valutazione dell'esercizio delle funzioni modificate.
I test sono stati eseguiti sulla mia macchina con CPU AMD Ryzen 5 7600X 6 core e GPU 4070 ti super.

Versione OpenMP

Considerando la seguente versione seriale:

```
int skyline( const points_t *points, int *s )
{
   const int D = points->D;
   const int N = points->N;
   const float *P = points->P;
   int r = N;
   ...
   for (int i=0; i<N; i++) {
      if ( s[i] ) {
         for (int j=0; j<N; j++) {
            if ( s[j] && dominates( &(P[i*D]), &(P[j*D]), D ) )  {
               s[j] = 0;
               r--;
            }
         }
      }
   }
   return r;
}
```

Nei due loop annidati è notabile da subito che non esistono dipendenze tra le iterazioni: ogni thread può elaborare un sottoinsieme di punti in modo indipendente, e l'ordine di esecuzione delle iterazioni non influisce sul risultato finale. L'unica criticità riguarda l'accesso concorrente all'array s ma la race condition è benigna e quindi non necessità l'utilizzo di operazioni atomiche. Diverso è il caso del contatore r, per il quale sarà necessario utilizzare la clausola *reduction* per garantire correttezza.

Nonostante entrambi i loop siano parallelizzabili indipendentemente, collassare con la clausola *collapse(2)* non sarebbe opportuno poiché si spezza l'ordine dei controlli, il che potrebbe creare loop sovrapposti dove l'eliminazione si esegue "fuori ordine" rischiando di lasciare sullo skyline dei punti che non dovrebbero rimanere per via che i punti che li dominano potrebbero essere già stati scartati dallo skyline.

Presumendo sempre che l'insieme contenente i punti non sia ordinato, qualunque sia l'input, e considerando un forte sbilanciamento del carico di lavoro; è meglio considerato usare un partizionamento irregolare a grana fine e bilanciare con la clausola *schedule(dynamic)*.

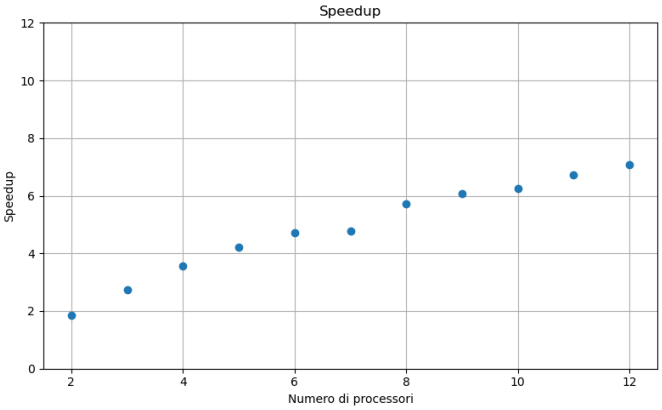
L'approccio che si sceglie è dunque definire la zona parallela prima dei cicli annidati, parallelizzare il ciclo esterno con scheduling dinamico a grana fine. In questa maniera manteniamo il bilanciamento del carico di lavoro e i test per valutare se un punto è ancora sullo skyline, riducendo il numero di iterazioni necessarie ma come compromesso si introduce lavoro aggiuntivo che può essere ammortato dallo scheduler dinamico.

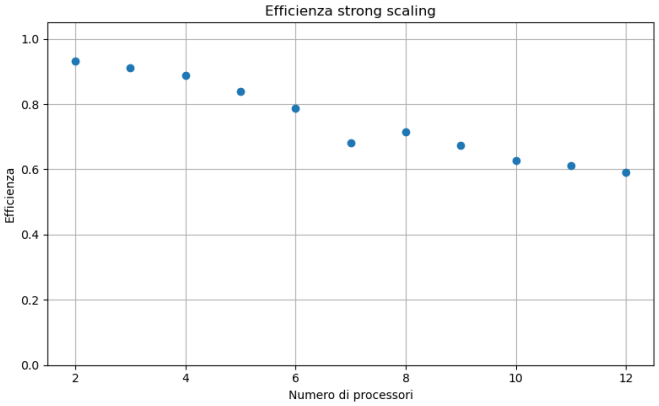
Nel calcolare lo strong scaling della soluzione implementata, sia dato un dataset con N=100000, D=10, senza punti dominanti; con i seguenti dati:

Sia lo speedup calcolato come: S(p) = T(1) / T(p)

Sia l'efficienza calcolata come: E(p) = S(p) / p

N° processori	Tempo (s)	Speedup	Efficienza
1.0	67.233	1.0	1.0
2.0	36.088	1.863	0.932
3.0	24.563	2.737	0.912
4.0	18.911	3.555	0.889
5.0	16.006	4.2	0.84
6.0	14.212	4.731	0.788
7.0	14.072	4.778	0.683
8.0	11.752	5.721	0.715
9.0	11.075	6.071	0.675
10.0	10.729	6.266	0.627
11.0	9.998	6.725	0.611
12.0	9.499	7.078	0.59



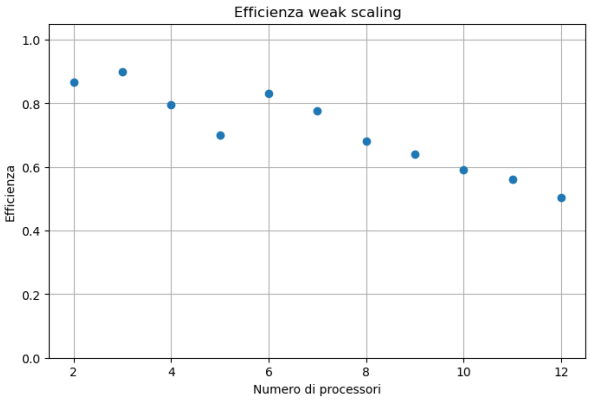


Per il weak scaling, siano dati più dataset senza punti dominanti, con D=10, N variabile, mantenendo un numero di operazioni proporzionale al numero di processori p e scheduling dinamico con dimensione dei blocchi pari a 1000; con i seguenti dati:

Si calcola la grandezza degli input come: N = ceil(5000 * sqrt(p))

Sia l'efficienza calcolata come: W(p)= T(1) / T(p)

N° processori	Input N	Tempo (s)	Efficienza
1.0	5000.0	0.163	1.0
2.0	7072.0	0.188	0.867
3.0	8661.0	0.181	0.9006
4.0	10000.0	0.205	0.7951
5.0	11181.0	0.233	0.6996
6.0	12248.0	0.196	0.8316
7.0	13229.0	0.21	0.7762
8.0	14143.0	0.239	0.682
9.0	15000.0	0.255	0.6392
10.0	15812.0	0.276	0.5906
11.0	16584.0	0.29	0.5621
12.0	17321.0	0.324	0.5031



L'implementazione mostra uno speedup ragionevole (≈x7) con 12 thread, anche se ben lontano da un'ideale x12, con un andamento sub-lineare che potrebbe deteriorare

velocemente; a parità di dimensioni N e D si deduce che avremo uno speedup maggiore o uguale 7.

Per l'efficienza, sia il strong che il weak scaling mostrano una rapida diminuzione dell'efficienza fino al ~50% - ~60% a 12 processori, dimostrando un valore alquanto povero nel caso peggiore dell'algoritmo; il calo iniziale dell'efficienza del weak scaling è dovuto alla grandezza della granularità e dell'input;

Possiamo considerare che un fattore determinante sulle prestazioni sia la scarsa località dei dati nel loop interno per ogni thread e la condivisione di lettura e scrittura dello skyline.

Il programma mostra tempi di esecuzione comunque ragionevoli con una scalabilità lontana dall'essere considerata buona.

Versione CUDA

In CUDA si adotta un parallelismo 1-D del loop interno a grana fine, in cui ogni thread elabora un singolo punto candidato dello skyline. L'indice globale del thread viene calcolato come:

$$j = blockIdx.x \times blockDim.x + threadIdx.x$$

Ogni thread copia le coordinate del proprio punto Pj in registri locali per ridurre gli accessi in memoria globale; successivamente confronta Pj con tutti gli altri punti Pi, con $i \in [0, N-1]$. Se Pj risulta dominato da un qualsiasi Pi, il thread interrompe subito il ciclo e marca Pj come non appartenente allo skyline, aggiornando l'array s[].

Per il conteggio dei punti sullo skyline si utilizza una strategia di riduzione che utilizza una variabile condivisa nei blocchi che ogni thread incrementa e il thread 0 dei ogni blocco sommano la variabile in un contatore globale.

Il calcolo delle prestazioni dell'implementazione CUDA sarà valutato tramite throughput, con complessità $O(N^2 * D)$ per un calcolo più approssimativo mantenendo costante D=10, utilizzando i Gops/sec come metrica; con la seguente formula:

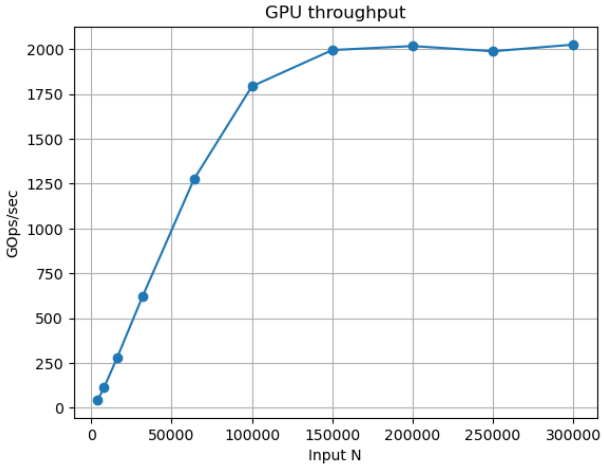
tempo (sec) = tempo di esecuzione (millisec) / 1000

n° operazioni (ops) = $N^2 * D$

operazioni al secondo (ops/sec) = n° operazioni / tempo

Giga Operazioni al secondo (gops/sec) = operazioni al secondo / 10^9

Input N	GOps/sec
4000.0	44.09
8000.0	113.3
16000.0	277.06
32000.0	622.17
64000.0	1275.16
100000.0	1792.55
150000.0	1994.73
200000.0	2017.05
250000.0	1987.99
300000.0	2025.1



Dai risultati ottenuti, le prestazioni crescono rapidamente fino a un input di dimensione $N \approx 100k$, mentre per input maggiori le prestazioni con all'incirca ~ 2000 Gops/sec, segnando il raggiungimento della saturazione della larghezza di banda della GPU;

La scalabilità è buona è buona fino a una certa soglia ma limitata; il maggiore ostacolo sono le letture non ottimizzate dei punti Pi.

Conclusioni

Sia l'approccio OpenMP sia l'approccio CUDA riescono a sfruttare in modo efficace il parallelismo ma soffrono entrambi dei problemi di accesso concorrente alla memoria, limitandone l'efficienza e scalabilità.

In generale, entrambi i modelli confermano che l'algoritmo skyline scala male per dataset di grandi dimensioni a causa della complessità $O(N^2)$. Miglioramenti significativi richiedono una migliore ottimizzazione dell'uso della memoria.